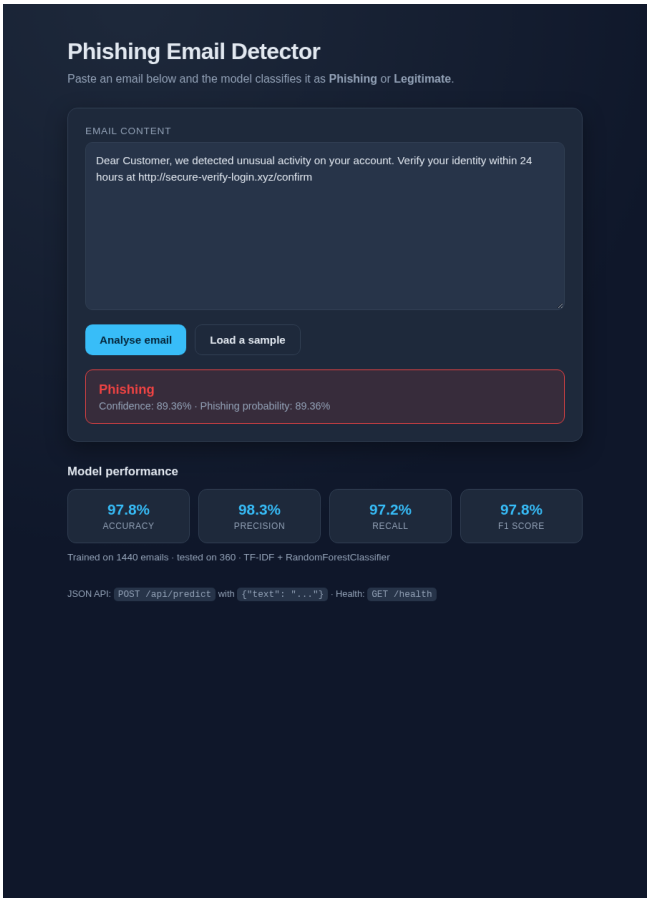
Phishing Email Detection Using NLP

Machine-learning web app that classifies email as phishing or legitimate

Sunny Gouria · January 2025 · Python · Flask · scikit-learn · NLP

Overview

This project is a phishing-email detector: a Natural Language Processing pipeline that reads an email and flags it as **Phishing** or **Legitimate**, served through a Flask web application. It uses TF-IDF vectorization (word and character n-grams) feeding a Random Forest classifier. On the bundled held-out test set the model reaches **97.8% accuracy**, with 98.3% precision and 97.2% recall on the phishing class — the metrics that matter for a security filter, since a detector that misses phishing is worse than useless.



The web app returning a verdict with confidence and live model metrics.

Model performance

Metrics are computed on a 20% held-out split of the dataset (1440 training emails, 360 test emails) and written to `model/metrics.json` on every training run.

Metric	Value	Why it matters
Accuracy	**97.8%**	Overall share of correct predictions.
Precision (phishing)	**98.3%**	Of emails flagged, the share that really are phishing.
Recall (phishing)	**97.2%**	Of real phishing emails, the share caught.
F1 score (phishing)	**97.8%**	Balance of precision and recall.

Note: the bundled dataset is a small synthetic sample built to train and run offline. Swap in a public corpus (Enron-Spam, SpamAssassin, the Nazario corpus, or the Kaggle phishing dataset) to train a production-grade model; the pipeline and code stay the same.

How it works

1. **Text preparation.** The subject and body are combined into one text field.
2. **Feature extraction.** TF-IDF converts text into weighted n-grams: word n-grams (1–2) capture phrases such as “verify your account”, while character n-grams (3–5) catch obfuscated tokens and odd URLs, and keep the model robust to unseen words.
3. **Classification.** A Random Forest (400 trees) predicts the probability that an email is phishing.
4. **Serving.** Flask exposes the trained pipeline through a web UI and a JSON API. The whole pipeline is saved with joblib, so training and serving stay consistent.

Tech stack

Layer	Tools
Language	Python 3.10+
ML / NLP	scikit-learn, TF-IDF, Random Forest
Data	pandas, numpy
Web	Flask, HTML, CSS, JavaScript
Persistence	joblib

Getting started

The project trains itself on first run, so three commands are enough to launch it.

1. Create and activate a virtual environment: `python -m venv .venv`, then activate it.
2. Install dependencies: `pip install -r requirements.txt`.
3. Start the app: `python app.py`, then open `http://127.0.0.1:5000`.

To retrain and print the full evaluation report:

```
python scripts/generate_sample_data.py   # only if data/emails.csv is missing
python train_model.py
```

API endpoints

Endpoint	Method	Purpose
/	GET	Web UI
/predict	POST	Form-based prediction
/api/predict	POST	JSON prediction — {"text": "..."}
/health	GET	Health check

Example: `curl -X POST http://127.0.0.1:5000/api/predict -H "Content-Type: application/json" -d '{"text": "Urgent: verify your account at http://secure-login.xyz"}'`

Project summary for a résumé

- Built an NLP phishing-email classifier that flags emails as phishing or legitimate using TF-IDF vectorization and a Random Forest model.
- Automated email triage for security teams, reducing manual filtering effort.
- Shipped a Flask web application serving real-time predictions through a UI and a JSON API.
- Reported precision, recall and F1 alongside accuracy, since a phishing filter's value depends on catching phishing without drowning users in false alarms.

Repository contents

app.py (Flask app) · train_model.py (training and evaluation) · predict_cli.py (command-line predictor) · scripts/generate_sample_data.py (sample dataset) · templates/ and static/ (UI) · tests/test_app.py (smoke tests) · requirements.txt · README.md · LICENSE (MIT).